

12-AI-CODING-STANDARDS.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-CODING-STANDARDS.md définit les standards obligatoires de production de code pour Vitala.

Il s'applique à :

- Frontend ;
- Backend ;
- Services ;
- API ;
- Database ;
- Scripts ;
- Tests ;
- Automatisations.

Toute IA produisant du code Vitala doit respecter ce document.

2. Principes fondamentaux du code

Le code Vitala doit être :

- lisible ;
- simple ;
- prévisible ;
- sécurisé ;
- maintenable ;
- testable ;
- évolutif.

La quantité de code produite n'est jamais un objectif.

La qualité du code est prioritaire.

3. Règles générales obligatoires

L'IA DOIT :

- écrire du code compréhensible ;
- utiliser des noms explicites ;
- éviter les raccourcis incompréhensibles ;
- respecter l'architecture ;
- supprimer le code mort.

L'IA NE DOIT PAS :

- produire du code temporaire non documenté ;
 - copier du code inutilement ;
 - utiliser des noms vagues.
-

4. Convention de nommage

Fichiers

Les noms doivent être explicites.

Correct :

```text id="3c4z7s" UserProfile.tsx`

`FlashEditor.tsx`

`sync-service.ts`

Incorrect :

```text id="bx8t1w"`

`Component1.tsx`

`NewFile.ts`

`test2.ts`

5. Composants React

Les composants doivent :

- utiliser PascalCase ;

- représenter une responsabilité claire.

Exemple :

```
```text id="c9jq9f" FlashCard.tsx
```

RadarMap.tsx

UserAvatar.tsx

Interdit :

```
```text id="6j2x8a"  
EverythingComponent.tsx
```

6. Fonctions

Les fonctions doivent :

- avoir un nom décrivant l'action ;
- être courtes ;
- avoir un comportement prévisible.

Exemple :

```
createFlashDraft()  
  
syncPendingOperations()  
  
validateUserProfile()
```

Éviter :

```
processData()  
  
handleStuff()  
  
doAction()
```

7. Variables

Les variables doivent être explicites.

Correct :

```
userProfile  
  
flashDraftId  
  
pendingSyncOperations
```

Incorrect :

```
data  
  
item  
  
value
```

sauf contexte évident.

8. TypeScript obligatoire

Tout nouveau code doit utiliser TypeScript correctement.

L'IA doit :

- définir les types ;
- éviter `any` ;
- utiliser les interfaces lorsque nécessaire.

Interdit :

```
const data:any
```

sauf justification documentée.

9. Organisation du code

Un fichier doit rester concentré sur son rôle.

Éviter :

- fichiers trop longs ;
- plusieurs responsabilités ;
- logique mélangée.

Si un fichier devient complexe :

l'IA doit proposer une séparation.

10. Gestion des erreurs

Les erreurs doivent être traitées explicitement.

Interdit :

```
try {  
  
}  
  
catch {  
  
}
```

sans gestion.

Obligatoire :

- capturer l'erreur ;
 - fournir un contexte ;
 - gérer le comportement attendu.
-

11. Commentaires dans le code

Les commentaires doivent expliquer :

- pourquoi une décision existe ;
- une logique complexe ;
- une contrainte importante.

Ils ne doivent pas expliquer une évidence.

Mauvais :

```
// Ajouter 1  
counter++
```

Bon :

```
// Conservation de l'ancien index pour maintenir  
// la compatibilité avec les synchronisations offline
```

12. Logique métier

La logique métier ne doit pas être placée :

- dans les composants UI ;
- dans les fichiers de configuration ;
- dans les utilitaires génériques.

Elle doit être dans :

- services ;
 - domaines ;
 - modules appropriés.
-

13. Gestion des états

L'IA doit éviter :

- états dupliqués ;
- états inutiles ;
- logique complexe dans les composants.

Chaque état doit avoir une raison claire.

14. Performance

L'IA doit considérer :

- rendu inutile ;
- appels réseau excessifs ;
- requêtes lourdes ;
- taille des bundles.

Optimiser uniquement lorsque nécessaire.

Pas d'optimisation prématurée.

15. Sécurité dans le code

L'IA ne doit jamais :

- exposer des secrets ;
- faire confiance aux entrées utilisateur ;
- contourner les permissions.

Toute donnée externe doit être validée.

16. Code Database

Les requêtes doivent :

- être sécurisées ;
- être optimisées ;
- respecter les modèles officiels.

Interdit :

- SQL non contrôlé ;
 - accès sans permission ;
 - logique métier cachée dans des requêtes.
-

17. Code API

Chaque endpoint doit avoir :

- validation entrée ;
 - réponse typée ;
 - gestion erreur ;
 - sécurité.
-

18. Code IA

Toute intégration IA doit préciser :

- modèle ;
 - objectif ;
 - contexte envoyé ;
 - limites ;
 - traitement des erreurs.
-

19. Revue automatique avant livraison

Avant de considérer le code terminé :

L'IA doit vérifier :

☐ Nommage correct

☐ Architecture respectée

☐ Types corrects

☐ Erreurs gérées

☐ Sécurité vérifiée

☐ Tests présents

☐ Documentation mise à jour

20. Règle finale

Le code Vitala doit rester compréhensible par une IA et un humain plusieurs années après sa création.

Un code qui fonctionne mais qui est incompréhensible est considéré comme incomplet.

Historique

Version	Modification
V1.0	Création des standards de code IA

Fin de AI/12-AI-CODING-STANDARDS.md